

Lab 20 — Continuous ADC: DMA-Backed Acquisition, Buffer Sizing and Overrun Recovery

ESP32-S3 FIRMWARE LAB SERIES · PHASE 3 · GAP-CLOSERS · GAP-CLOSER

Estimated time	2–3 hrs
Proves	DMA acquisition, pool sizing, overrun detection and recovery
Prerequisites	Labs 4 and 8.
Beyond the board	A potentiometer, or a jumper to 3V3 / GND.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **\$20.1 objective**, **\$20.2 parts and wiring**, **\$20.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept, then **Phase 0 configure**, **Phase 1 the complete code**, walkthrough, **Phase 2 run**, **Phase 3 prove it**, worked extensions.

One note on scope, because the title is precise on purpose: the continuous-ADC driver allocates and walks the DMA descriptors for you. What you own here is the frame and pool policy, the completion callback and the overrun response — which is where the real engineering decisions are.

20.1 Objective & TM4C bridge

Set up a DMA-driven acquisition path directly, rather than getting DMA for free inside a driver. Labs 9 and 22 use DMA, but the driver hides it entirely. Here it is still the driver that owns the descriptors and the DMA channel — what you own is the frame and pool policy, the completion callback, and the overrun response. **This closes a genuine gap** — "I have used DMA" and "I have configured DMA" are different claims, and interviewers can tell them apart in one question.

TM4C BRIDGE

A bare-metal ADC lab moves every sample with the CPU, polling the conversion-complete flag. That works at low rates and collapses at high ones. DMA is the answer to "how do I sample at 20 kHz without spending all my CPU on it," and the TM4C had a μ DMA controller for exactly the same reason.

20.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	Required	The ADC is on-chip.
Potentiometer or a jumper to 3V3/GND	Required	Something to read on GPIO1. Even a bare wire you touch gives a varying signal.

Wire the potentiometer's outer legs to 3V3 and GND and its wiper to **GPIO1** (ADC1 channel 0). If you have no pot, jumper GPIO1 to 3V3 for a full-scale reading and to GND for zero — enough to prove the path works.

WARNING**Use ADC1, not ADC2**

On the ESP32 family, **ADC2 is shared with the Wi-Fi radio**. If Wi-Fi is active, ADC2 reads fail or return garbage. ADC1 is always yours. This is a documented restriction that catches people whose sensor readings mysteriously break the moment they add networking, and it is exactly the kind of chip-specific detail worth knowing.

20.3 How it works

You have used DMA already — the SPI driver and the display driver both use it internally. This lab is about configuring an acquisition path yourself, because "I have used DMA" and "I have configured DMA" are different claims and an interviewer can tell them apart in one question.

What DMA actually is

Direct Memory Access is a small dedicated hardware engine that moves blocks of data between a peripheral and memory *without the CPU touching each byte*. You tell it where from, where to, and how much; it performs the transfer and interrupts you when the block is finished.

It is a separate piece of silicon that shares the memory bus with the CPU. That sharing is the one cost: while DMA is moving data, it occasionally steals a bus cycle from the processor. On this chip that is negligible, but on very tight systems "bus contention" is a real term you will meet.

The arithmetic that makes the case

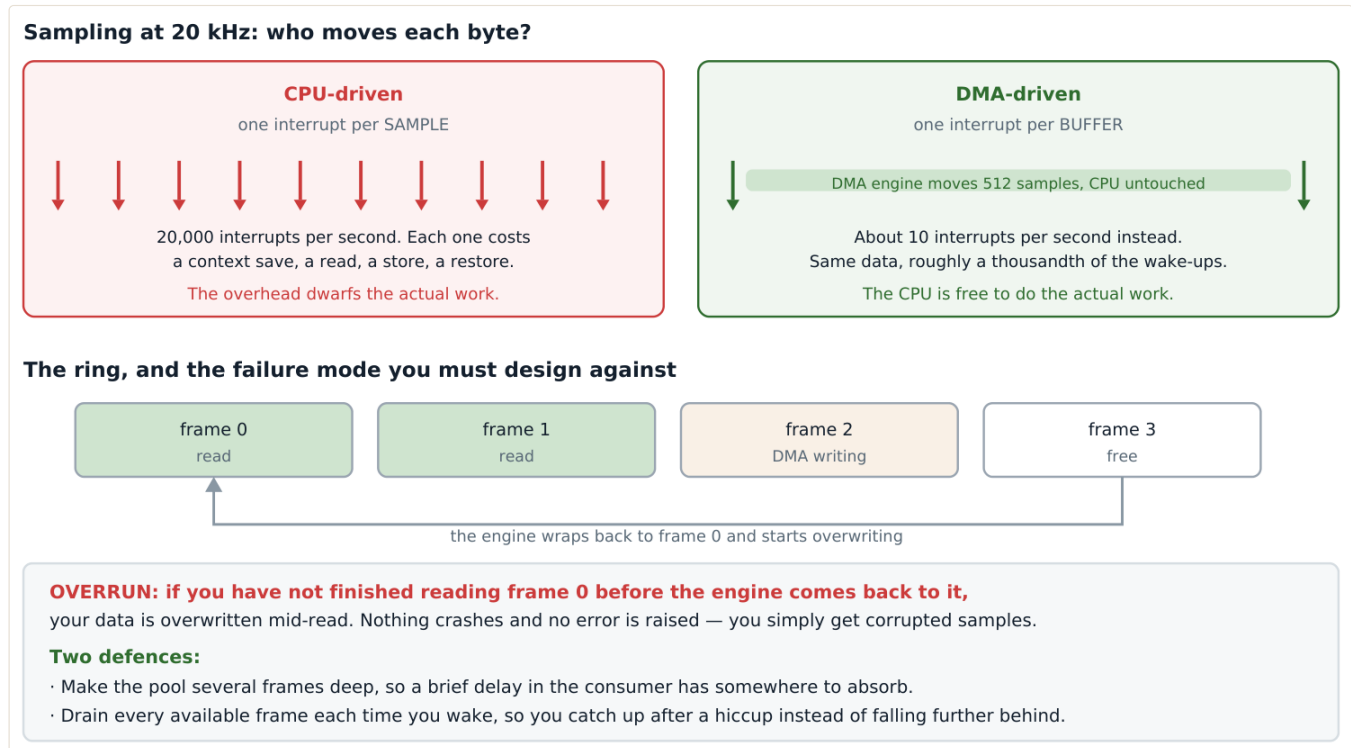


Figure 20.1. Top: the same 20 kHz acquisition, done two ways. Bottom: the ring the DMA engine fills, and the overrun condition — the characteristic DMA bug, which corrupts data silently rather than crashing.

CPU-driven versus DMA-driven acquisition, and the ring buffer overrun condition.

Sampling at 20 kHz, who moves each byte?

CPU-driven means one interrupt per SAMPLE: 20,000 interrupts per second, each costing a context save, a read, a store and a restore. The overhead dwarfs the actual work.

DMA-driven means one interrupt per BUFFER: the DMA engine moves 512 samples with the CPU untouched, giving about 10 interrupts per second instead. Same data, roughly a thousandth of the wake-ups, and the CPU is free to do the actual work.

The ring, and the failure mode you must design against. Four frames in rotation: frame 0 (read), frame 1 (read), frame 2 (DMA currently writing), frame 3 (free). When the engine finishes frame 3 it wraps back to frame 0 and starts overwriting.

OVERRUN: if you have not finished reading frame 0 before the engine comes back to it, your data is overwritten mid-read. Nothing crashes and no error is raised; you simply get corrupted samples.

Two defences: - Make the pool several frames deep, so a brief delay in the consumer has somewhere to absorb. - Drain every available frame each time you wake, so you catch up after a hiccup instead of falling further behind.

Sampling an ADC at 20 kHz, CPU-driven, means **20,000 interrupts per second**. Each one costs a context save, a register read, a store, and a context restore — call it a few microseconds. The

overhead of handling the samples ends up dwarfing the work of actually using them, and the CPU has little time left for anything else.

DMA-driven, the engine writes each conversion straight into a buffer and interrupts you once per **buffer** — with **256** samples per frame — 1024 bytes divided by the 4 bytes each conversion result occupies — which at 20 kHz is **78.125** frames per second. Same data, and the interrupt count falls by a factor of exactly 256, which is the samples-per-frame number itself.

That is the entire value proposition, and it is why DMA is not optional for displays, audio, cameras or fast ADCs. It is also why you should *measure* the difference in §20.7 rather than take it on trust.

Descriptors, and how continuous acquisition works

The DMA engine knows nothing about your program. It follows a small data structure called a **descriptor** that says, in effect, "write this many bytes starting here." Chain descriptors together and the engine walks the chain, which is how acquisition becomes continuous rather than one-shot.

The classic arrangement is **ping-pong**, or double buffering: while the engine fills buffer B, you process buffer A; when it finishes B it moves back to A, and by then you should be done with it. That alternation is what makes the capture genuinely gapless — there is never a moment where samples arrive with nowhere to go.

ESP-IDF's continuous ADC driver generalises this to a **ring** of several frames, which is the same idea with more headroom.

Overrun: the characteristic DMA bug

If you have not finished with a frame before the engine wraps around to it, your data is overwritten *while you are reading it*. Nothing crashes. It does not crash, and if you have not asked to be told it is easy to miss — you simply get samples that are part old and part new. But **an error IS available**: ESP-IDF provides an `on_pool_ovf` callback, and `adc_continuous_read()` returns `ESP_ERR_INVALID_STATE` when the pool filled and samples were lost. The listing in §20.5 registers both, which is why this lab can count overruns rather than guess at them.

That silence is what makes overrun nasty, and it is why the two defences matter:

Make the pool several frames deep. A pool of exactly one frame means *any* delay in the consumer corrupts data. Several frames give a brief hiccup somewhere to be absorbed.

Drain everything each time you wake. Loop on the read call until it reports nothing left, rather than taking one frame per wake-up. That way a delay costs you latency once instead of putting you permanently behind.

DMA-capable memory, and why a plain array will not do

The engine accesses memory directly, bypassing the CPU's cache and its view of the address space. So a DMA buffer has to satisfy constraints an ordinary variable does not: it must live in a region the engine can physically reach, and it must meet alignment requirements.

That is why DMA buffers are allocated with `heap_caps_malloc(size, MALLOC_CAP_DMA)` rather than declared as a plain array. Hand the engine an ordinary buffer and the transfer fails outright or, worse, silently writes somewhere it should not.

In this lab the driver allocates the ring for you, so you specify sizes rather than pointers — but the constraint is the same underneath, and you meet it directly in Lab 22 where you allocate the display stripe yourself.

Two chip-specific facts worth knowing

ADC2 is shared with the Wi-Fi radio. If Wi-Fi is active, ADC2 reads fail or return garbage. ADC1 is always yours. This is documented, and it catches people whose perfectly good sensor readings break the moment they add networking — a genuinely confusing symptom if you do not know the rule.

Attenuation sets the input range. The ADC compares against an internal reference, and an attenuator scales the input down so you can measure a wider range. `ADC_ATTEN_DB_12` gives roughly the full 0 to 3.3 V span; lower attenuation gives a narrower range with finer resolution over it. Choosing it wrong means your signal either clips or uses a fraction of the available codes.

Also worth knowing: the ESP32 family's ADC is not especially linear, and Espressif provides a **calibration** API that converts raw counts to millivolts using per-chip factory data burned into eFuse. For anything where absolute accuracy matters, you use it.

20.4 Phase 0 — configure and wire

0.1 — project files

```
// sdkconfig.defaults
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_FREERTOS_HZ=1000
```

```
// top-level CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab20_adc_dma)
```

```
// main/CMakeLists.txt
idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")
```

0.2 — wiring

Signal	Pin	Note
ADC input	GPIO1 (ADC1 channel 0)	A potentiometer wiper, or just a jumper to 3V3 or GND for a fixed level

WARNING

ADC1 only — and this is stronger than "prefer ADC1"

You may have read that ADC2 is merely "shared with the Wi-Fi radio." For **continuous mode on the ESP32-S3 the driver refuses outright**, logging *"ADC2 continuous mode is no longer supported, please use ADC1"* and returning an error unless you deliberately set a Kconfig override. So for this lab ADC1 is not the preference, it is the only option.

0.3 — the sizing arithmetic, worked

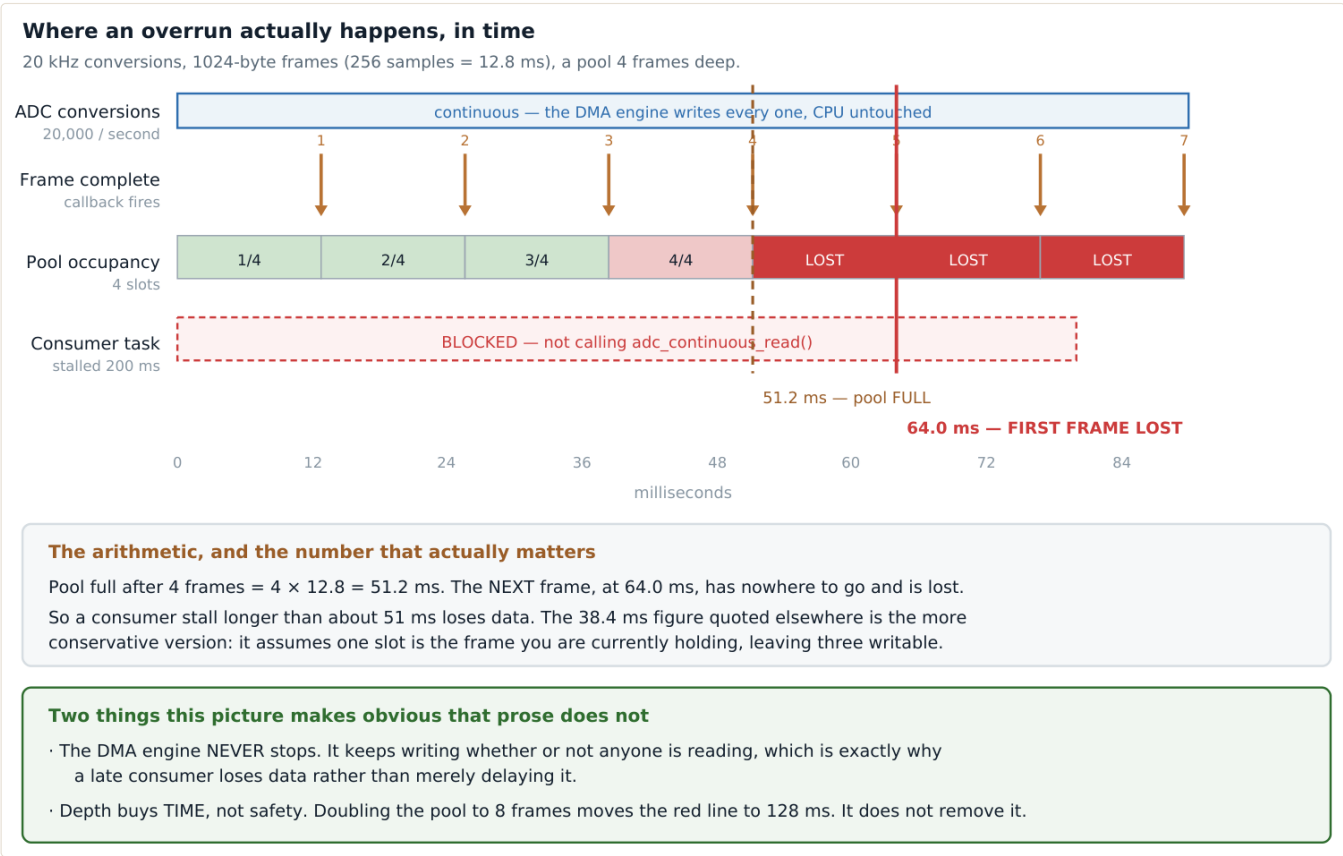


Figure 20.2. [DERIVED] The same numbers as a timeline. A stalled consumer fills the pool at 51.2 ms; the frame due at 64.0 ms has nowhere to go and is lost. Every value here follows from the three constants below.

Where an ADC pool overrun happens, in time. [DERIVED] Timeline spanning 0-90 ms. Configuration: 20 kHz conversions, 1024-byte frames (256 samples = 12.8 ms each), pool 4 frames deep.

Four stacked rows:

Row	What it shows
ADC conversions	One continuous band across the whole timeline. The DMA engine writes every conversion, CPU untouched.
Frame complete	Downward arrows at 12.8, 25.6, 38.4, 51.2, 64.0, 76.8 ms - one callback per frame
Pool occupancy	Blocks reading 1/4, 2/4, 3/4, 4/4, then LOST, LOST, LOST in red
Consumer task	A dashed red band labelled BLOCKED - not calling <code>adc_continuous_read()</code>

Two vertical markers: **51.2 ms - pool FULL**, and **64.0 ms - FIRST FRAME LOST**.

The arithmetic: the pool fills after 4 frames = $4 \times 12.8 = 51.2$ ms. The next frame, due at 64.0 ms, has nowhere to go and is lost. So a consumer stall longer than about 51 ms loses data. The 38.4 ms figure quoted elsewhere in the lab is the conservative version: it assumes one slot is the frame you are currently holding, leaving three writable.

Read the timeline before the arithmetic, because it makes two things obvious that the numbers alone do not.

The DMA engine never stops. The top band is continuous. It keeps writing whether or not anyone is reading, which is precisely why a late consumer *loses* data rather than merely delaying it. Compare a queue between two tasks, where a slow consumer just makes the producer block — here the producer is hardware and it cannot be told to wait.

Depth buys time, not safety. Doubling the pool to eight frames moves the red line from 64 ms to 128 ms. It does not remove it. There is no pool size that makes overrun impossible, only sizes that make it improbable for a consumer whose worst case you have actually measured — which is why §20.8 has you measure it from both sides rather than trust the number.

Three constants decide everything, and they are the real engineering content of this lab. Here is the derivation rather than the numbers alone:

```
SOC_ADC_DIGI_RESULT_BYTES (ESP32-S3) = 4 bytes per conversion result

samples per frame = FRAME_BYTES / 4 = 1024 / 4           = 256 samples
frame period      = 256 / 20000 Hz                       = 12.80 ms
frame rate       = 20000 / 256                           = 78.125 frames/s
pool depth       = POOL_BYTES / FRAME_BYTES = 4096/1024 = 4 frames
overrun margin   = 3 spare frames x 12.80 ms             = 38.4 ms
```

NOTE

Two slack figures appear in this lab. They are both right.

Figure	Assumes	Use it for
51.2 ms	All four slots are writable — the consumer is fully stalled and holds nothing.	The absolute point at which data is first lost. What the timeline above shows.
38.4 ms	One slot is the frame you are currently holding, leaving three writable.	The conservative budget to design against, because a real consumer <i>is</i> holding a frame while it works.

Design to the smaller one and measure against the larger. Both are **models of the pool, not guarantees published by the driver** — which is why §20.8 brackets the real margin experimentally from both sides instead of trusting either.

Sizing alternatives, for when you need to change it:

frame	pool	samples/frame	rate	period	slack
1024 B	4096 B	256	78.1/s	12.8 ms	38.4 ms
2048 B	8192 B	512	39.1/s	25.6 ms	76.8 ms

frame	pool	samples/frame	rate	period	slack
1024 B	8192 B	256	78.1/s	12.8 ms	89.6 ms

Bigger frames mean fewer interrupts but more latency before you see any given sample. A deeper pool costs only RAM and buys slack directly. If your consumer's worst case might be 50 ms, row one is too shallow and now you know before it bites.

20.5 Phase 1 — the complete main.c

```
// =====
// Lab 20 -- Continuous ADC with DMA-backed acquisition and overrun recovery
// Complete file. Wire a pot wiper (or a jumper) to GPIO1.
// =====
#include <stdio.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_continuous.h"
#include "esp_timer.h"
#include "esp_log.h"

#define SAMPLE_HZ    20000
#define FRAME_BYTES  1024                // 256 samples -> 12.80 ms
#define POOL_BYTES   (FRAME_BYTES * 4)  // 4 frames -> 38.4 ms of slack
#define STALL_MS     0                  // set to 200 for proof case 2

static const char *TAG = "lab20";

static adc_continuous_handle_t adc;
static TaskHandle_t consumer;

// ---- counters: ONE WRITER EACH. This matters more than it looks. ----
// An earlier edition incremented a single g_overruns from BOTH the ISR and
// the task. Two defects in one line: the same overflow episode was counted
// twice, and counter++ is a read-modify-write, so two contexts doing it can
// lose an update. 'volatile' prevents caching, not interleaving.
// Single-writer variables need no lock, and the two numbers mean different
// things -- how many overflow EVENTS fired, vs how many reads FOUND one.
static volatile uint32_t g_frames;      // task only
static volatile uint32_t g_samples;    // task only
static volatile uint32_t g_bad_meta;    // task only
static volatile uint32_t g_ovf_cb;     // ISR only
static volatile uint32_t g_ovf_read;   // task only

// ----- callbacks
static bool IRAM_ATTR on_conv_done(adc_continuous_handle_t h,
                                   const adc_continuous_evt_data_t *e, void *u)
{
    BaseType_t hp = pdFALSE;
    vTaskNotifyGiveFromISR(consumer, &hp); // signal only. Move NO data here.
    return hp == pdTRUE;                   // the driver yields for us
}

static bool IRAM_ATTR on_pool_ovf(adc_continuous_handle_t h,
                                   const adc_continuous_evt_data_t *e, void *u)
```

```

{
    g_ovf_cb++;                                // ONLY this context writes this
    return false;                             // no logging here -- IRAM-safe means tiny
}

// ----- consumer
static void consumer_task(void *arg)
{
    (void)arg;
    uint8_t frame[FRAME_BYTES];
    uint32_t got = 0;
    int64_t t0 = esp_timer_get_time();

    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

#ifdef STALL_MS
        vTaskDelay(pdMS_TO_TICKS(STALL_MS)); // proof case 2: force an overrun
#endif
        // DRAIN. A 'while', not an 'if'. One frame per wake-up would leave you
        // permanently a frame behind after any hiccup, and the backlog grows.
        for (;;) {
            esp_err_t err = adc_continuous_read(adc, frame, FRAME_BYTES,
                                                &got, 0); // 0 = do not wait

            if (err == ESP_OK) {
                g_frames++;
                int n = got / SOC_ADC_DIGI_RESULT_BYTES;
                uint32_t sum = 0;
                int valid = 0;

                for (int i = 0; i < n; i++) {
                    adc_digi_output_data_t *s = (adc_digi_output_data_t *)
                        &frame[i * SOC_ADC_DIGI_RESULT_BYTES];
                    // Validate rather than assume. With one pattern configured
                    // every sample should match; a mismatch is a real fault.
                    if (s->type2.unit != ADC_UNIT_1 || s->type2.channel != ADC_CHANNEL_0) {
                        g_bad_meta++;
                        continue;
                    }
                    sum += s->type2.data;
                    valid++;
                }
                g_samples += valid;

                if (valid && (g_frames % 78) == 0) { // ~1 Hz at 78.125 fps
                    int64_t dt = esp_timer_get_time() - t0;
                    ESP_LOGI(TAG,
                        "frames=%" PRIu32 " rate=%.1f/s mean=%" PRIu32
                        " ovf_cb=%" PRIu32 " ovf_read=%" PRIu32 " bad=%" PRIu32,
                        g_frames, g_frames * 1e6 / (double)dt,
                        sum / valid, g_ovf_cb, g_ovf_read, g_bad_meta);
                }
            }
            else if (err == ESP_ERR_INVALID_STATE) {
                // OVERRUN. The pool filled and samples were LOST. Distinct
                // from "nothing to read", and it needs the opposite response.
                g_ovf_read++;
                ESP_LOGW(TAG, "pool overflow -- samples lost "
                    "(events=%" PRIu32 " reads=%" PRIu32 ")",

```

```

        g_ovf_cb, g_ovf_read);
        break;
    }
    else if (err == ESP_ERR_TIMEOUT) {
        break; // ring empty: normal, go wait again
    }
    else {
        ESP_LOGE(TAG, "unexpected read error: %s", esp_err_to_name(err));
        break; // never swallow an unknown error
    }
}
}

// ----- app_main
void app_main(void)
{
    xTaskCreate(consumer_task, "adc", 4096, NULL, 5, &consumer);

    adc_continuous_handle_cfg_t pool = {
        .max_store_buf_size = POOL_BYTES,
        .conv_frame_size    = FRAME_BYTES,
    };
    ESP_ERROR_CHECK(adc_continuous_new_handle(&pool, &adc));

    adc_digi_pattern_config_t pattern = {
        .atten    = ADC_ATTEN_DB_12, // widest range, roughly 0..3.3 V
        .channel  = ADC_CHANNEL_0,   // GPIO1
        .unit     = ADC_UNIT_1,      // ADC1 -- ADC2 continuous is unsupported
        .bit_width = ADC_BITWIDTH_12, // 4096 codes
    };
    adc_continuous_config_t cfg = {
        .sample_freq_hz = SAMPLE_HZ,
        .conv_mode       = ADC_CONV_SINGLE_UNIT_1,
        .format          = ADC_DIGI_OUTPUT_FORMAT_TYPE2,
        .pattern_num     = 1,
        .adc_pattern     = &pattern,
    };
    ESP_ERROR_CHECK(adc_continuous_config(adc, &cfg));

    adc_continuous_evt_cbs_t cbs = {
        .on_conv_done = on_conv_done,
        .on_pool_ovf  = on_pool_ovf, // register BOTH, or overrun is invisible
    };
    ESP_ERROR_CHECK(adc_continuous_register_event_callbacks(adc, &cbs, NULL));
    ESP_ERROR_CHECK(adc_continuous_start(adc)); // takes the HANDLE

    ESP_LOGI(TAG, "running: %d Hz, %d B frames, %d B pool (%d frames, ~%.1f ms slack)",
        SAMPLE_HZ, FRAME_BYTES, POOL_BYTES,
        POOL_BYTES / FRAME_BYTES,
        (POOL_BYTES / FRAME_BYTES - 1) *
        (FRAME_BYTES / SOC_ADC_DIGI_RESULT_BYTES) * 1000.0 / SAMPLE_HZ);
}

```

20.6 Code walkthrough, the parts that matter

on_conv_done moves no data

Notice what the callback does **not** do: it does not read, copy or process a single byte. All the data movement was done by the DMA engine before this ran. The callback exists purely to say "a frame is ready."

`vTaskNotifyGiveFromISR` is a **direct-to-task notification** — lighter than a semaphore because each task has a built-in notification value, so no kernel object is allocated. It targets one specific task, which is fine here.

The `return hp == pdTRUE` is `portYIELD_FROM_ISR` expressed as a return value: you tell the *driver* whether a context switch is needed and it performs it after its own cleanup. Callback-based drivers commonly do this.

on_pool_ovf — the callback the original lab never registered

Its absence is why an earlier edition could claim "overrun raises no error." It had not asked to be told. The callback must be **IRAM-safe**, which is why it does nothing but increment a counter — no logging, no allocation.

NOTE

On IRAM_ATTR, more precisely than the usual shorthand

The common claim is "an ISR must be in IRAM or it crashes." That is too strong. ESP-IDF's actual behaviour is that non-IRAM-safe handlers are **deferred while the flash cache is unavailable** — delayed, not fatal. The stricter placement rules bite when you enable

`CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE`, and then *every function the handler calls and every datum it touches* must also be in internal memory, not just the handler itself. Marking one function `IRAM_ATTR` is necessary, not sufficient.

the five separate counters

The concurrency point in the comment is the important one. `g_ovf_cb` counts overflow *events* raised by hardware; `g_ovf_read` counts reads that *found* the pool overflowed. One episode is typically seen by both, so a single shared counter reported roughly double the real figure — and, worse, was a genuine data race between ISR and task. **One writer per counter** makes each safe without a lock.

the drain loop is a while, not an if

The second overrun defence. Looping drains every frame queued while you were busy. Reading one frame per wake-up means that after any hiccup you are permanently one frame behind and the backlog only grows. The zero timeout is what lets the loop terminate.

three distinct error branches

The single most valuable correction in this lab. The original exited on *any* non-`ESP_OK`, conflating `ESP_ERR_TIMEOUT` ("nothing to read right now" — normal, happens every cycle) with `ESP_ERR_INVALID_STATE` ("the pool overflowed and your data is gone"). Those need opposite responses, and a third branch catches anything unexpected rather than silently treating it as normal.

validating s->type2.unit and .channel

Each result carries which ADC unit and channel produced it, because a multi-channel scan interleaves results. With one pattern every sample should match, so a mismatch means something is genuinely wrong. Two comparisons turn a silent corruption into a counted one.

20.7 Phase 2 — build and run

```
idf.py build
idf.py -p COM6 flash monitor
```

KEY POINT

Expected output

```
I lab20: running: 20000 Hz, 1024 B frames, 4096 B pool (4 frames, ~38.4 ms slack)
I lab20: frames=78 rate=78.0/s mean=2047 ovf_cb=0 ovf_read=0 bad=0
I lab20: frames=156 rate=78.1/s mean=2048 ovf_cb=0 ovf_read=0 bad=0
```

The rate must sit near 78/s, not 10/s. If it is much lower, frames are being dropped and the overflow counters will say so. `mean` tracks the pot: roughly 0 at GND, roughly 4095 at 3V3.

20.8 Phase 3 — prove it

#	Do this	Expected
1	Run as printed for 60 s.	rate ~78/s, all three of <code>ovf_cb</code> , <code>ovf_read</code> , <code>bad</code> stay 0 .
2	Force an overrun. Set <code>STALL_MS</code> to 200 — well past the 38.4 ms of slack. Rebuild.	Both overflow counters climb and the warning fires. This validates the slack model from §20.4.
3	Set <code>STALL_MS</code> back to 0.	Counters stop climbing and the rate recovers without a reboot .
4	Set <code>STALL_MS</code> to 30 (just under 38.4).	Should <i>not</i> overrun, or only rarely. This brackets the real margin from both sides.
5	Set <code>PPOOL_BYTES</code> to <code>FRAME_BYTES * 8</code> and repeat case 2 with <code>STALL_MS</code> 200.	Slack becomes 89.6 ms, so a 200 ms stall still overruns. Depth buys margin, it does not remove the limit.

KEY POINT

Cases 2 and 4 together are the debug-log entry

"I calculated 38.4 ms of consumer slack from the frame size and pool depth, then verified it: a 30 ms stall did not overrun and a 200 ms stall did." That is a derived number confirmed by experiment from both directions, which is a genuinely strong thing to be able to say.

20.9 Extensions, worked

20.9.1 — What DMA actually saved you

NOTE

How numbers here are labelled

[DERIVED] arithmetic, shown so you can check it. **[MEASURE]** you must run it and record your own value. Only **[MEASURE]** results are yours to claim anywhere public.

The headline claim is 20,000 interrupts per second against about 78. Rather than assert it, count them. Add to the conversion callback:

```
static volatile uint32_t g_irq;    // ISR only
// inside on_conv_done():
    g_irq++;
```

and print it in the periodic log.

KEY POINT

Result and the arithmetic

	Interrupts/ s	Per sample cost	CPU spent on delivery
CPU-driven [DERIVED] , not measured	20,000	~2 µs context save/ restore	~4% of one core, doing nothing useful
DMA-driven [MEASURE] — read your own log	~78	same ~2 µs	~0.016%

A factor of 256, which is exactly `FRAME_BYTES / SOC_ADC_DIGI_RESULT_BYTES` — the interrupt reduction is the samples-per-frame number. That relationship is worth stating in an interview because it shows you understand where the saving comes from rather than that DMA is "faster."

Note the CPU-driven row is calculated, not measured, because this driver has no polling mode. Say so if you quote it.

20.9.2 — Reading actual millivolts

Raw codes are fine for detecting change but meaningless as a voltage: the ESP32 ADC is not especially linear, and every chip differs. Espressif burns per-chip calibration into eFuse at manufacture. Use it:

```
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"

static adc_cali_handle_t cali;

static void cali_init(void)
{
    adc_cali_curve_fitting_config_t c = {
```

```

        .unit_id = ADC_UNIT_1,
        .chan    = ADC_CHANNEL_0,
        .atten    = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_12,
    };
    if (adc_cali_create_scheme_curve_fitting(&c, &cali) == ESP_OK)
        ESP_LOGI(TAG, "calibration available");
    else
        ESP_LOGW(TAG, "no calibration on this chip -- raw codes only");
}

// then, where you log the mean:
int mv = 0;
if (cali && adc_cali_raw_to_voltage(cali, sum / valid, &mv) == ESP_OK)
    ESP_LOGI(TAG, "mean %" PRIu32 " raw = %d mV", sum / valid, mv);

```

Expect roughly 3100-3200 mV at 3V3 rather than a clean 3300 — the 12 dB attenuator saturates a little below the rail. That is a real characteristic worth knowing, and it is exactly why calibration exists.

20.9.3 — Scanning two channels

The pattern array is plural for a reason. Add a second channel and the hardware interleaves results, tagging each with its channel — which is what the validation code in §20.5 was already prepared for.

```

adc_digi_pattern_config_t pattern[2] = {
    { .atten = ADC_ATTEN_DB_12, .channel = ADC_CHANNEL_0,
      .unit = ADC_UNIT_1, .bit_width = ADC_BITWIDTH_12 }, // GPIO1
    { .atten = ADC_ATTEN_DB_12, .channel = ADC_CHANNEL_1,
      .unit = ADC_UNIT_1, .bit_width = ADC_BITWIDTH_12 }, // GPIO2
};
cfg.pattern_num = 2;
cfg.adc_pattern = pattern;

```

Then split the accumulation by `s->type2.channel` instead of rejecting non-zero channels.

WARNING

The catch worth knowing

`sample_freq_hz` is the **total** conversion rate across all channels, not per channel. With two patterns at 20 kHz you get 10 kHz each. Getting this wrong is a classic way to end up with half the sample rate you designed for and no error to tell you.

20.10 Docs and references

Topic	Where
Continuous ADC driver, callbacks, overflow	ESP-IDF v5.5 → Peripherals → ADC → Continuous mode
ADC calibration schemes	ESP-IDF v5.5 → Peripherals → ADC Calibration
IRAM-safe interrupt requirements	ESP-IDF v5.5 → System → Interrupt allocation

Topic	Where
ESP32-S3 ADC errata	Espressif ESP32-S3 errata sheet — explains the ADC2 continuous restriction

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

NOTE

Already fixed in the lab above — nothing to do

This section is the engineering record: what the earlier version of this lab got wrong, how it was found, and what replaced it. The lab as printed above is the corrected version. It is kept because the method used to catch the error is worth more than the error itself.

Lab 20 — a compile error, wrong arithmetic, and a wrong claim about overrun

WARNING

Four confirmed defects.

1. It does not compile. The listing calls `adc_continuous_start()` with no arguments. The API is `adc_continuous_start(adc_continuous_handle_t handle)`. Corrected:

```
ESP_ERROR_CHECK(adc_continuous_start(adc));
```

2. The sample arithmetic is wrong. On the ESP32-S3 one conversion result is **4 bytes** (`SOC_ADC_DIGI_RESULT_BYTES`). So a 1024-byte frame holds **256 samples**, not 512, and at 20 kHz that is **78.1 frames per second** with a 12.8 ms frame period — not the "roughly 10 frames per second" §20.7 tells you to expect. Both the figure and the verification text carried the wrong numbers.

3. Overrun does report an error. §20.3 says "Nothing crashes. No error is raised." That is false. ESP-IDF provides an `on_pool_ovf` callback, and `adc_continuous_read()` returns `ESP_ERR_INVALID_STATE` when the pool has filled and samples were lost. The listing registers only `on_conv_done` and silently exits its read loop on *any* non-`ESP_OK` result, conflating a normal empty-ring timeout with a genuine overflow.

4. The title overclaims. `adc_continuous_new_handle()` allocates and manages the descriptors and DMA channel itself. You choose frame and pool sizes and consume frames; you never touch a descriptor. "DMA You Configure Yourself" is not what the lab does.

NOTE

And one correction the audit understated

§20.3 says ADC2 is "shared with the Wi-Fi radio" and fails when Wi-Fi is active. For **continuous mode on the ESP32-S3 the situation is stronger than that**: the driver refuses outright. ESP-IDF's `adc_continuous.c` logs "ADC2 continuous mode is no longer supported, please use ADC1. Search for errata on espressif website for more details" and returns an error unless you deliberately set `CONFIG_ADC_CONTINUOUS_FORCE_USE_ADC2_ON_C3_S3`. So for this lab the rule is not "prefer ADC1" — it is "ADC1 is the only option."

Fix: Lab 20 is retitled "**Continuous ADC with DMA-Backed Acquisition, Buffer Sizing and Overrun Recovery**" and supplies the corrected listing with overflow counting and error discrimination.

Framework version note

NOTE

ESP-IDF v6.0 is now the stable release

These labs are written and validated against **ESP-IDF v5.5**, and pinning a version is the right call for reproducibility — every listing here was checked against it. But v5.5 should no longer be described as the current starting point. **ESP-IDF v6.0 has shipped** and carries breaking changes and removals.

Keep building this track on v5.5 so the listings match. Then, as a genuinely useful final exercise, port your integrated project to v6.0 using Espressif's migration guide and record every API and configuration change you had to make. Version migration is ordinary professional work and almost nobody has done it before their first job.

One concrete example of the drift: v6.0 adds `adc_continuous_parse_data()` and `adc_continuous_read_parse()`, which do the per-sample decoding that Lab 20 does by hand.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 20.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20 ←	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.